

Soft-Margin SVM: Slack, Hinge Loss and C

Contents

1. Overview	2
2. Why the hard margin is not enough	2
3. From maximising to minimising	3
4. Slack: how far a point breaks the rules	3
4.1 The new term	3
4.2 What ξ measures	4
5. The soft-margin loss	5
6. The hyperparameter C	6
7. Comparison with logistic regression	7
8. Why “soft margin”	8
9. Summary	9
10. Key terms	9

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Classification problems ([Note 3](#)); Dot product ([Note 48](#)); Regularisation ([Note 63](#)); Equation of a hyperplane ([Note 70](#)); Perceptron trick ([Note 71](#)).
- **Compare with:** Log loss (binary cross entropy) ([Note 73](#)); Logistic regression ([Note 75](#)).

1. Overview

Key point: Soft-margin SVM lets some points break the rules, but charges for it. Its loss adds a margin error and C times a classification error; C sets the balance.

The hard-margin SVM of the [SVM maths Note](#) needs data that a hyperplane separates perfectly. Real data almost never is. The **soft-margin SVM** fixes this by giving every point a little room to be wrong, and adding the cost of that room to the loss (Figure 1).

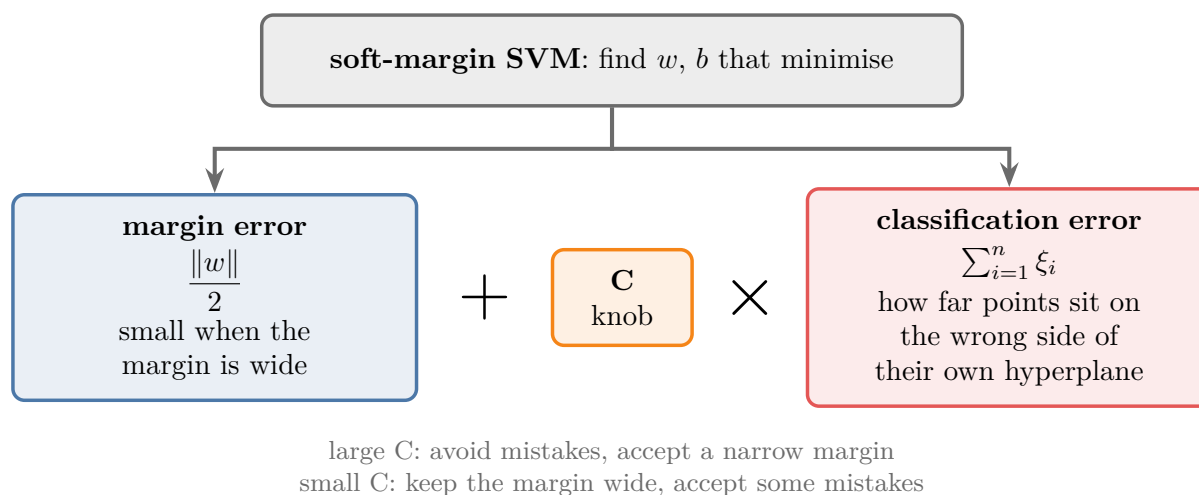


Figure 1: The soft-margin loss: a margin error plus C times a classification error

2. Why the hard margin is not enough

Key point: The hard-margin constraints allow no exceptions, so a single point on the wrong side leaves the problem without a solution.

The hard-margin SVM solves:

$$\arg \max_{w, b} \frac{2}{\|w\|} \quad \text{such that} \quad y_i (w^T x_i + b) \geq 1 \quad \text{for all } i$$

The constraint says that every green point lies on or above π^+ and every red point on or below π^- . A green point below π^+ breaks it: its label is $+1$ but $w^T x_i + b$ is less than 1, so the product is less than 1. A red point above π^- breaks it the same way, with $y_i = -1$ and $w^T x_i + b > -1$.

This is a constrained optimisation problem with n constraints, one per point. It assumes that **no** point ever lies inside the margin or on the wrong side. Real datasets are at best almost linearly separable: a few points, often outliers, sit on the wrong side. With such data the constraints cannot all be met (Figure 6 of the SVM maths Note), so the formulation fails in most real situations.

We need a version that leaves some space for outliers. That version is the soft-margin SVM.

3. From maximising to minimising

Key point: Maximising $2/\|w\|$ is the same as minimising $\|w\|/2$. The minimising form is easier to extend.

Before changing anything, we rewrite the hard-margin problem. Maximising a positive function f gives the same answer as minimising $1/f$: a quantity is largest exactly where its inverse is smallest. So:

$$\arg \max_{w,b} \frac{2}{\|w\|} = \arg \min_{w,b} \frac{\|w\|}{2}$$

With numbers, for the best line of the SVM intuition Note, $\|w\| = 0.899$: the margin $2/0.899 = 2.22$ is as large as possible exactly when $0.899/2 = 0.45$ is as small as possible. This is still the hard-margin SVM; only the form has changed, because a minimisation is easier to add terms to.

Extra: In practice, and in scikit-learn, the term is written $\frac{1}{2}\|w\|^2$ instead of $\|w\|/2$. Squaring does not change which w is smallest, and the squared form is smooth (it has no kink at $w = 0$) and is exactly the L2 penalty of [Ridge regression](#).

4. Slack: how far a point breaks the rules

Key point: Each point gets a slack ξ_i . It is 0 for a point on the correct side of its own hyperplane, and grows with how far the point sits on the wrong side of it.

4.1 The new term

Key point: Add $\sum \xi_i$ to the loss: the total amount by which the points break their constraints.

To make the SVM soft, we add one more term to the minimisation:

$$\arg \min_{w, b} \frac{\|w\|}{2} + \sum_{i=1}^n \xi_i$$

The Greek letter ξ is “xi” (some texts use ζ , “zeta”). Each training point i gets its own value ξ_i , called its **slack**. The algorithm now looks for the w and b that make **both** terms small.

4.2 What ξ measures

Key point: During training, green points are judged against π^+ and red points against π^- , not against π .

During training, each class is judged against its **own** hyperplane: green points against π^+ , red points against π^- . (The middle line π is only used later, to classify new points.)

- A green point on or above π^+ , or a red point on or below π^- , is fine: $\xi_i = 0$.
- A green point below π^+ , or a red point above π^- , is an error point. Its ξ_i is its distance from its own hyperplane, measured in the wrong direction.

So an error point may be inside the margin but still on the correct side of π , or right across π on the wrong side. The further it is from where it should be, the larger its ξ_i .

Figure 2 shows the two error points of a soft-margin SVM. The green point (6, 4.3) lies below π^+ , even below π : $\xi = 1.33$. The red point (2.8, 5.2) lies above π^- and above π : $\xi = 1.32$. Every other point has $\xi = 0$.

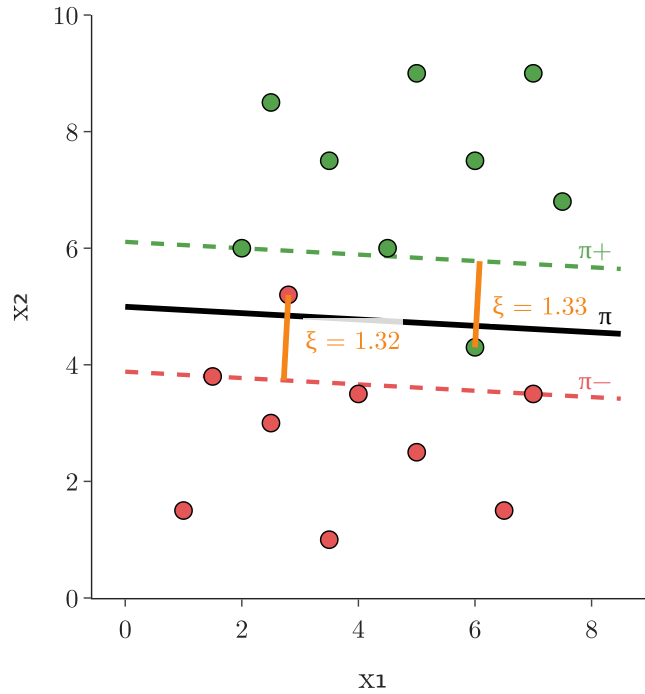


Figure 2: The slack ξ of the two points that break their constraints (orange); all other points have $\xi = 0$

Minimising $\sum \xi_i$ therefore means minimising these distances: keeping the errors few and small.

Extra: Precisely, the slack is $\xi_i = \max(0, 1 - y_i(w^T x_i + b))$, and the hard constraint is loosened to $y_i(w^T x_i + b) \geq 1 - \xi_i$ with $\xi_i \geq 0$. This ξ_i is measured in the units of the margin, where the distance from π to π^+ counts as 1. The distance in the plot is $\xi_i / \|w\|$: for the green point, $1.33 / 0.899 = 1.48$. A point with $0 < \xi_i \leq 1$ is inside the margin but still correctly classified; $\xi_i > 1$ means it is on the wrong side of π .

5. The soft-margin loss

Key point: Loss = margin error + $C \times$ classification error. The first term wants a wide margin, the second wants few and small mistakes.

The two terms pull in different directions:

- $\|w\|/2$ is the **margin error**. It equals $1/d$, so it falls as the margin widens: a wide margin gives a small margin error.
- $\sum \xi_i$ is the **classification error**. It falls as the error points move back to their own side.

The last step is a weight on the classification error, a positive number C . In words: find the

line that balances a wide margin against small errors, with C deciding how much the errors count. As a formula, the soft-margin SVM solves:

$$\arg \min_{w,b} \underbrace{\frac{\|w\|}{2}}_{\text{margin error}} + C \underbrace{\sum_{i=1}^n \xi_i}_{\text{classification error}}$$

With numbers, for the line of Figure 2 with $C = 1$: $\|w\|/2 = 0.899/2 = 0.45$ and $\sum \xi_i = 1.33 + 1.32 = 2.65$, so the loss is $0.45 + 1 \times 2.65 = 3.10$. The same mistakes with $C = 10$ would cost $0.45 + 26.5 = 26.95$, which is why a larger C pushes the line to fix them.

6. The hyperparameter C

Key point: Large C : avoid mistakes, accept a narrow margin. Small C : keep the margin wide, accept some mistakes. Tune C with cross-validation.

C is a [hyperparameter](#): a tuning knob we set before training. It can be any positive number.

- **Large C** (say 1,000 or 10,000): the classification error dominates. The algorithm stops caring about the width of the margin and concentrates on misclassifying no point.
- **Small C** (say 0.1, 0.01 or 0.001): the margin error dominates. The algorithm keeps the margin wide even if some points end up misclassified.

Figure 3 shows the same data with three values of C . With $C = 1000$ the margin is only 0.37 wide, but no point is misclassified. With $C = 1$ it is 2.22 wide with two mistakes; with $C = 0.05$ it is 4.65 wide.

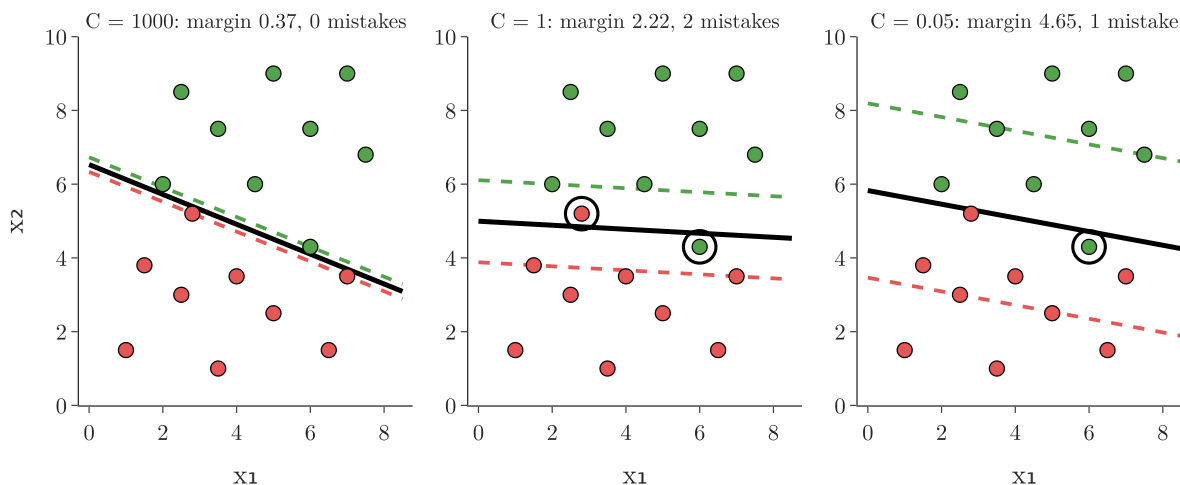


Figure 3: Large C : a narrow margin and no mistakes. Small C : a wide margin and some mistakes (ringed)

Neither extreme is right in general. A model with no training mistakes and a thin margin

may be fitting outliers; a model with a very wide margin may ignore real structure. So we pick C for each dataset with cross-validation, usually through a grid search over values such as 0.001, 0.01, ..., 1,000.

Python: In scikit-learn, C is an argument of `SVC`; its default is 1.

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

svm = SVC(kernel="linear", C=1)      # soft margin, C = 1
svm.fit(X, y)

# choose C by 5-fold cross-validation
grid = GridSearchCV(SVC(kernel="linear"),
                    {"C": [0.001, 0.01, 0.1, 1, 10, 100, 1000]}, cv=5)
grid.fit(X, y)
grid.best_params_
```

7. Comparison with logistic regression

Key point: Both are “loss + regularisation”. SVM uses the hinge loss where logistic regression uses the log loss, and in both, C is the inverse of the regularisation strength λ .

The soft-margin loss has the same shape as the loss of regularised logistic regression:

	Error on the data	Regularisation	Strength
Logistic regression	log loss	$\lambda \ w\ ^2$ (L2)	λ on the penalty
Soft-margin SVM	$\sum \xi_i$: the hinge loss	$\ w\ /2$	C on the error

- The margin term $\|w\|/2$ plays the role of the **regularisation** term: it keeps w small, which here means a wide margin.
- The error term $\sum \xi_i$ is called the **hinge loss**, the SVM’s counterpart of the log loss.
- C multiplies the error instead of the penalty, so it works the other way round: C is inversely proportional to λ . A large C means weak regularisation; a small C means strong regularisation.

This is also why scikit-learn’s `LogisticRegression` has a C and no λ (see the [logistic regression hyperparameters Note](#)). The convention comes from SVM: a larger C means less regularisation in both.

Extra: Written with $z = y_i(w^T x_i + b)$, the hinge loss of one point is $\max(0, 1 - z)$ and the log loss is $\log(1 + e^{-z})$ (the same log loss as before, with labels ± 1). Figure 4 compares them. The hinge loss is exactly 0 for every point beyond its own hyperplane ($z \geq 1$), so those points do not affect the line at all; only points on or inside the margin matter, which is why SVM depends only on its support vectors. The log loss never quite reaches 0, so every point keeps pulling a little.

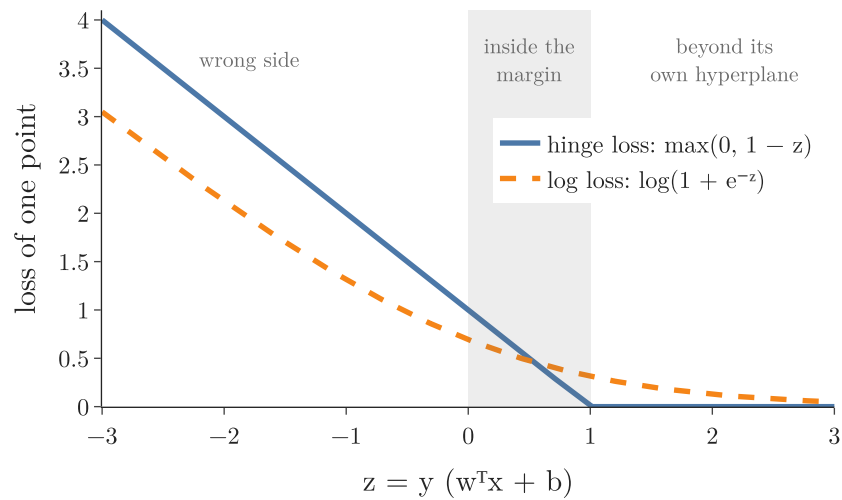


Figure 4: Hinge loss and log loss for one point, against $z = y(w^T x + b)$

8. Why “soft margin”

Key point: The margin is “soft” because points may enter it or cross it, at a price.

The hard-margin SVM forbids any point inside the margin. The soft-margin SVM relaxes that rule: it allows some errors, as long as it pays for them through $C \sum \xi_i$. Because the criterion is softened, this formulation is called the **soft-margin SVM**. It is the version scikit-learn’s SVC uses.

9. Summary

	Hard-margin SVM	Soft-margin SVM
Objective	$\min \ w\ /2$	$\min \ w\ /2 + C \sum \xi_i$
Points inside the margin or misclassified	not allowed	allowed, at a cost ξ_i
Works on	perfectly separable data	almost separable data with outliers
Knob	none	C

- $\arg \max 2/\|w\|$ equals $\arg \min \|w\|/2$.
- Slack ξ_i : 0 on the correct side of a point's own hyperplane, otherwise its distance from that hyperplane.
- Loss = margin error + $C \times$ classification error (hinge loss).
- Large C: narrow margin, few mistakes. Small C: wide margin, more mistakes. Tune C by cross-validation.
- C plays the role of $1/\lambda$, as in `LogisticRegression`.

10. Key terms

Term	Meaning
Soft-margin SVM	The SVM that allows points inside the margin or on the wrong side, at a cost controlled by C
Slack (ξ)	How far a training point lies on the wrong side of its own hyperplane; 0 if it is on the correct side
Margin error	The term $\ w\ /2$ of the SVM loss; small when the margin is wide
Classification error (SVM)	The term $\sum \xi_i$ of the SVM loss: the total slack of all points
Hinge loss	$\max(0, 1 - y(w^T x + b))$ per point: the error term of the soft-margin SVM
C (SVM)	The weight on the classification error; a large C means few mistakes and a narrow margin, a small C a wide margin